Bottom-up Parsing

CS/COE 1622

Jarrett Billingsley

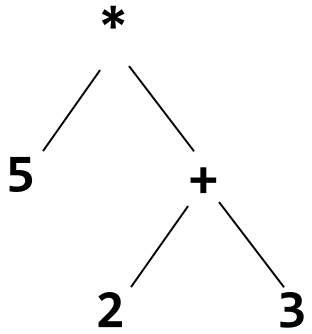
Class Announcements

- today we're finishing parsing!! yaaay
  - so after today you'll have everything you need for project 2!!!!!!
- bottom-up parsing is (imo) best used for parsing **expressions,** so we should start by talking about...

Operators

Evaluating (and producing) trees

- we saw before that given an AST, we could **evaluate it:**



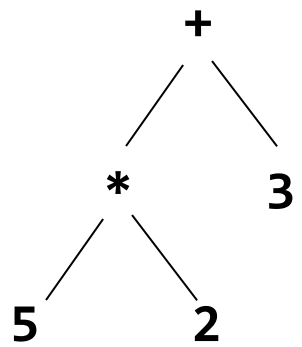
I think you would agree that the **order of operations** here is to do the addition, *then* the multiplication.

but we're talking about parsing now. **what source code** would I write to produce this AST?

```
5 * (2 + 3)
```

why did we have to put parentheses there?

because of the order of operations:
without them, `5 * 2 + 3` would parse as:



PEMDAS, BODMAS, BIDMAS…

- **officially,** the order of operations used by mathematicians is:
  1. parentheses/brackets
  2. exponentiation/roots
  3. multiplication/division
  4. addition/subtraction
- the order of the letters in the acronym (MD vs. DM) **does not say that e.g. multiplication happens before division.**
- when writing math horizontally like **`27 / 3 / 9`**, there's a problem:
  - even among mathematicians, **there is no consensus on what order horizontal division should be done!**
- in programming languages, *all* math is written horizontally…
  - and you have e.g. **`%`**, **`&&`**, **`||`**, **`&`**, **`|`**, **`^`**, **`!`**, **`~`**, **`<`**, **`>`**, **`<=`**, **`>=`**, **`==`**, **`!=`**, …
- so we need a **more rigorous way** of dealing with this.

Operator Precedence

- here's a more formalized, algorithmically-checkable method:
  - you **rank** the operators by giving each a number, its **precedence**
  - **that precedence** decides the order of evaluation
- another way of thinking of it is: if you wrote the expression *without parentheses,* the precedence tells you **where they *should* go:**

Operator	Prec
**	1st
* / %	2nd
+ -	3rd

```
a + b * c   ➡  a + (b * c)
a ** b * c  ➡  (a ** b) * c
```

but this still isn't enough information to disambiguate when operators of the **same precedence** appear next to each other:

```
27 / 3 / 9  ➡  (27 / 3) / 9  or
               27 / (3 / 9)  ?
```

Operator Associativity

- **in a precedence tie,** an operator's **associativity** breaks the tie.

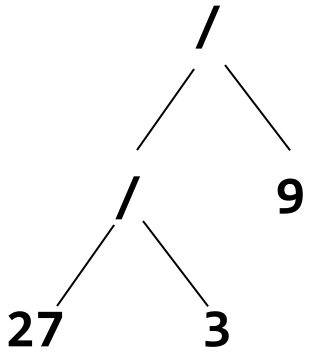
left-associative operators (e.g. **+**, **-**, *****, **/**, **%**) evaluate things **left-to-right**.

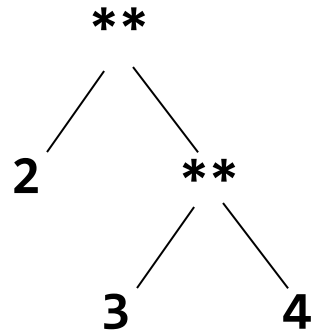
```
27 / 3 / 9
(27 / 3) / 9
```

and **right-associative** operators (e.g. ******, **=**) evaluate things **right-to-left.**

```
2 ** 3 ** 4
2 ** (3 ** 4)
```

in other words, associativity says **which way the tree branches.**





More about associativity

- to be clear: associativity only applies when you have **adjacent operators of the *same precedence*.**
  - `a ** b + c` parses as `(a ** b) + c`, because exponentiation's precedence ranks higher than addition's.
- it's also possible for operators to be **non-associative.**
  - that means it's an *error* for them to be adjacent.
- for example, what does `x < y < z` mean?
  - in Java, you get a **type error.**
  - in Python, it's parsed as `(x < y) && (y < z)`.
  - in the language we're designing, we *could* say that comparisons are **non-associative,** and block this potentially-confusing code as a **parsing error** instead.

Unary and postfix operators

- operators that take two operands are called **binary.** but many (especially in C-style languages) are **unary** and operate on one value.
  - `-x`, `~x`, `!x`, `&x`, `*x`, `++x`, `--x`, `(int)x`
  - these are given a precedence **higher than any binary operator.**
    - this matches our intuition: `-3 + 5` should give **2**, not -**8.**
  - these are all *right-associative,* but it doesn't come up much.
    - `-~x` would bitwise-complement x first, then negate that.
- finally there are **postfix operators** which come *after* their operand.
  - `x++`, `x--`, `a[i]`, `obj.field`, `f(x)`
  - and these have the **highest precedence of all!**
    - `-f(x)` calls the function first, *then* negates the return value.
  - these are all *left-associative,* and that's pretty common to see.
    - `f(x)[i].field` calls **f** first, then indexes the returned array, then accesses the field from the array value.

Summarizing operators

- a language's specification will typically have all the operators, their **precedences,** and their **associativities** listed in a table:

Operator	Prec	Assoc
`f() a[i] o.f`	1	L
`!x -x *x &x ++x --x`	2	R
`* / %`	3	L
`+ -`	4	L
`< <= > >=`	5	L
`== !=`	6	L
`&&`	7	L
`||`	8	L
`=`	9	R

this is a pretty typical arrangement for a C-style language (excluding bitwise operators, casts, reflexive assignments, and some other obscure operators)

but where are parentheses?

well, they're *not operators*. parentheses only control the order of evaluation and the way the AST is built; they don't *do* anything otherwise.

Building a grammar for expressions

A grammar from the bottom up

- rather than starting with the "overall" structure of an expression...
  - let's start with the **most basic parts** and gradually combine them.
- a **primary expression** is the most basic kind.

literals fall into this category.

```
318    "hello"
   9.5       'x'
```

identifiers are another kind: they name single things.

```
x      main
            System
```

last, we'll include **parenthesized expressions,** because they contain whole sub-expressions. so, our grammar is:

```
PrimaryExp: IdExp | IntLitExp | ParenExp
IdExp:      <identifier token>
IntLitExp:  <int literal token>
ParenExp:   '(' Exp ')'
```

(**Exp** is the top-level expression rule that we're building up to)

Terms: primaries with decorations

- since unary and postfix operators are so closely associated with their operands, we can define a **`Term`** rule like this:

```
Term: UnaryOp* PrimaryExp PostfixOp*
```

- that is, 0 or more unary operators before a primary "core," followed by 0 or more postfix operators.
- **`UnaryOp`** and **`PostfixOp`** might be defined like:

```
UnaryOp:   '-' | '!'
PostfixOp: CallOp | FieldOp | IndexOp
CallOp:    '(' ')' | '(' Exp (',' Exp)* ')'
FieldOp:   '.' <identifier token>
IndexOp:   '[' Exp ']'
```

- example productions of this grammar are **`x`**, **`-x`**, **`f()`**, **`!f(x).z`**, **`!!x`**

Expressions: the big picture

- really, every expression looks like this:

term *op* **term** *op* **term** *op* **term** *op* **term**

where each *op* is a binary operator.

so, we'll define these rules:

```
Exp:      Term (BinaryOp Term)*
BinaryOp: '+'|'-'|'*'|'/'|'%'|…etc.
```

but **what about the associativity and precedence??**

well, we just… uh… *won't encode that into the grammar.*

Wait. What?

- it is *absolutely* possible to encode precedence and associativity into the expression grammar, and then write a fully recursive-descent parser for expressions. (I know, because I've done it.)
- but the result is just… well. here's a simple mathematical language:

```
Exp: Add
Add: Mul (('+'|'-')  Mul)*
Mul: Neg (('*'|'/'|'%')  Neg)*
Neg: Pow | ('-' Neg)
Pow: Pri ('**' Pow)*
Pri: Id  | IntLit | '(' Exp ')'
```

is it obvious that this accurately captures all expressions?

can *you* identify the associativity and precedence of these operators?

what if, like C or C++, we had *15 precedence levels* and *dozens* of operators? how readable would that be?

So no! We won't do it!

- because we're defining our language's grammar a bit loosely, *it doesn't really matter* if our grammar is 100% accurate.
  - instead, the ambiguities in the grammar are resolved by the **table of precedence and associativity.**
- any complex idea can be expressed in multiple different ways.
  - unless you have some kind of externally-imposed reason to stick with *one* way of writing something…
  - it makes sense to use different ways according to their strengths.
- so…
  - how do we write an algorithm to *parse* this stuff??

Bottom-up parsing

The core of the algorithm

- let's assume we **only have left-associative binary operators.**
- so, to parse `Term (BinaryOp Term)*`, we can write something like:

```
let mut LHS = parse_term();
while current token is a binary operator {
  let op = current token;
  move to next token;
  let RHS = parse_term();
  LHS = AstNode::new(LHS, op, RHS);
}
return LHS;
```

- it might seem strange to replace the contents of the LHS variable inside the loop, but this is how it builds up the tree.

Okay, let's try it (animated)

- given this sequence of tokens, let's see what happens:

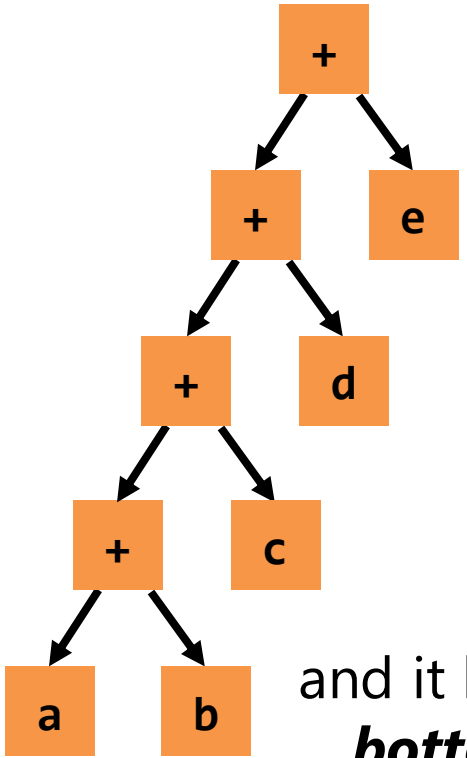
x + y - z *<eof>*

Token	Action	LHS	op	RHS
x	**LHS** = parse term	`x`		
+	**op** = +, loop!		+	
y	**RHS** = parse term **LHS** = (**LHS op RHS**)	`(x + y)`		y
-	**op** = -, loop!		-	
z	**RHS** = parse term **LHS** = (**LHS op RHS**)	`((x + y) - z)`		z
<eof>	stop looping!			

That's why it's called bottom-up (animated)

- this algorithm produces a tree that branches to the left, like this.

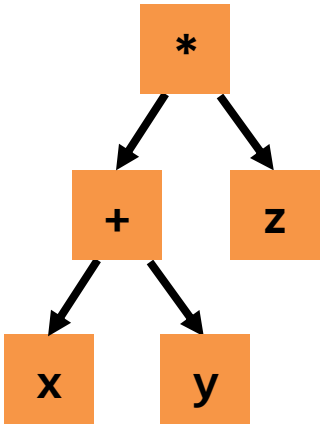
a + b + c + d + e



and it builds it…
bottom up!

but if we allow **all** the operators, what would be a **problem input** for this?

x + y * z



it was a **little too eager to produce (x + y),** when it didn't know that there was a * coming up.

Accounting for precedence

- we have to tweak our algorithm a bit.

```
let mut LHS = parse_term();
while current token is a binary operator {
  let op = current token;
  move to next token;
  let RHS = parse_term();
```

at this point, we don't know if RHS is *our* second operand, or if it's *the next* operator's *first* operand.

so, we'll have to check if the current token is a binary operator *whose precedence is **higher than op**'s.*

and if that's the case, we um. uh.

*wait, what **do** we do then?*

```
  LHS = AstNode::new(LHS, op, RHS);
}
```

Implicit parentheses

- if you think about what we're doing in a different way...

a +(b * c * d * e)+ f

we're sort of **inserting parentheses** as we go.

so when we see the first *, we should say "oh, the left parenthesis goes **before b...**

"...and the right parenthesis goes before the **first operator** that has a **lower precedence than multiplication."***

since we want to *put the parsing of + on hold,* the most natural way to parse the multiplications is to **recurse.**

(let's look at the example to see how this is done.)

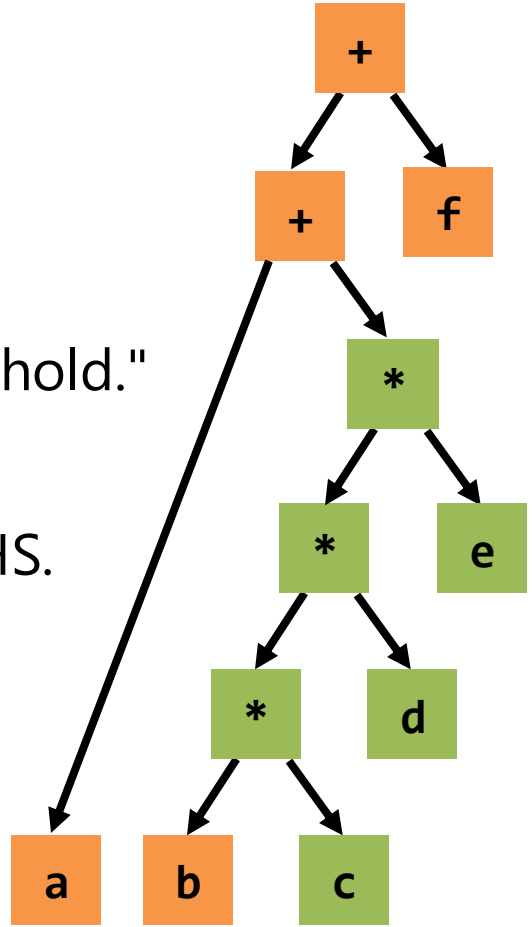
How that parses (animated)

- Looking at this input again, let's see how the AST is built for it:

a + b * c * d * e + f

1. **a** is parsed.
2. **b** is parsed.
3. ***** is seen, causing us to recurse. **+** is put "on hold."
4. the subtree of multiplications is parsed.
5. that subtree is returned and becomes **+**'s RHS.
6. the final **+ f** is parsed.

Right-branching is done by the **R**ecursion;
Left-branching is done by the **L**oop.



Finishing up by parsing terms

- finally, these are more straightforward.

  `Term: UnaryOp* PrimaryExp PostfixOp*`

- since the unary operators are **Right-associative...**
  - you got it: we have to **Recurse** to build the AST for those.
- after that, primary expressions are straightforward.
  - parenthesized expressions will work just fine, because `')'` is not an operator, and will cause the operator parsing loop to terminate.
- and finally, postfix expressions come after the primaries.
  - the primary has to be passed to the postfix parser, so that it can become the LHS of those postfix operators.
  - postfix operators are **Left-associative**, so we can use a **Loop**.
  - also, it's not an error to see 0 postfix operators. they're optional!

What about right-associative binary operators?

- right-associativity is another kind of **right-branching.**
  - so, we would have to change the **recursion** condition in the binary operator parsing.
- currently we recurse when we see an operator of **higher precedence.**
  - we will also recurse when we see a **right-associative operator** of the ***same or higher*** **precedence.**
- this is so if you have multiple right-associative operators in a row of the same precedence, you still get the right-branching structure.
- in `a ** b ** c`...
  - we see that the second `**` has the same precedence as the first
  - so we recurse, giving us an RHS of `b ** c`
  - ultimately giving us `a ** (b ** c)`

Things to think about

Bottom-up parsing without recursion

- by using auxiliary stack data structures, we *could* remove the need for recursion entirely.
- did any of you do the expression evaluation assignment in 445?
  - where you had an **operator stack** and an **operand stack?**
- if you still have that code, try going back and looking at it.
- what if the operand stack held **AST nodes,** instead of Doubles?
  - and when you handle names/operands, you **pushed AST nodes?**
- and in the parts where you **pop operators** and evaluate…
  - what if, instead of evaluating, you **created/pushed AST nodes?**
- *that's it, that's a parser, you already made most of an expression parser and you didn't even know it*

Can we parse *everything* with bottom-up parsing?

- ***Yes!***
- actually, generalized bottom-up parsing is **more powerful** than top-down parsing!
  - there are some grammars that top-down parsers can never parse.
- however…
  - once you move beyond expressions it gets stupidly complex
  - the complexity is not really justified for programming languages
    - their grammars are typically not that complicated
  - so it's mostly out of the scope of this course, IMO
- for our language, we can parse *everything* using a hybrid approach:
  - use **recursive descent** for the program's broad structure
  - use **bottom-up** for expressions

How does that work though?

- well, expressions only appear *inside* other pieces of code.
- so, a grammar might look like this:

```
Program:     Function*
Function:    'fn' Id '(' ')' Block
Block:       '{' Stmt* '}'
Stmt:        IfStmt | ExpStmt | AssignStmt
IfStmt:      'if' Exp Block ('else' Block)?
ExpStmt:     Exp ';'
AssignStmt:  Exp '=' Exp ';'
```

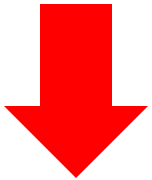
- all of *these* rules would be parsed with recursive descent…
- and whenever we need to parse an **Exp**, we use bottom-up parsing.

Syntactic Sugar

Yummy

- each language feature adds complexity in **every stage of the compiler:** lexing, parsing, semantics, optimization, code generation…
- so, one approach to adding features to our languages is by defining them in terms of **other, simpler features that already exist.**

```
let lhs = self.parse_term()?;
```



this is **syntactic sugar,** and it's called that because it usually makes your code easier to write.

```
let lhs = match self.parse_term() {
  Ok(x)  => x,
  Err(e) => return Err(e),
};
```

Another example

- Java's generic for loop is just sugar for using an iterator.

```
for(int i : someCollection) { … }
```

- becomes something like:

```
for(Iterator<Integer> iter = someCollection.iterator();
    iter.hasNext(); )
{
    int i = iter.next().intValue();
    …
}
```

How is it implemented?

- by **desugaring:** rewriting the AST after parsing.
  - we saw previously that trees are easy to create and manipulate.
- but we have to be a little careful.
  - up next is **semantic analysis,** and if we desugar the code before then, it might give really confusing errors about *code that the programmer didn't write!*
- but that's it for parsing!